



Université Cheikh Anta Diop de Dakar
Master 2 Business Intelligence — Cours de NLP

Répondre à des questions de santé en langues africaines

Challenge Zindi — TF-IDF, NLLB, mT5-small et Gemma E2B (QLoRA) comparés sur 5 langues

Andil Mchangama Nasma Youssouf M'madi Abdoulaye Diouf Harouna Mbaye Hamidou Mbodji

CONTEXTE

Une réponse médicale n'est utile que si elle est comprise

5

langues : anglais, akan, luganda, swahili, amharique

8

sous-ensembles pays/langue (ex. Amh_Eth)

39K

questions de santé au total (Train + Val + Test)

- Le challenge Zindi vise à répondre à des questions de santé **dans la langue locale** de l'utilisateur, pas seulement en anglais.
- Une bonne partie des populations concernées ne lit ni n'écrit l'anglais couramment — une réponse mal comprise perd son utilité, même juste cliniquement.

NOTRE DÉMARCHE

Trois approches, du plus simple au plus ambitieux

BASELINES ZINDI

- Retrieval **TF-IDF** et LLM **NLLB-200** — code fourni par l'organisateur, exécuté tel quel comme point de repère indépendant.

MODÈLE 1

- **mT5-small** (300M) en full fine-tuning — recommandé par Zindi pour un premier run rapide, abordable sur un seul GPU.

MODÈLE 2

- **Gemma E2B** (5,13Md) adapté par **QLoRA** — approche plus ambitieuse, budget de calcul bien plus élevé.

POINT DE DÉPART

Les baselines fournies par Zindi

BASELINE	ROUGE-1	ROUGE-L	CONSTAT
TF-IDF retrieval	0,4207	0,3655	Score élevé mais trompeur — recopie une réponse humaine, ne génère rien
NLLB zero-shot	0,0074	0,0071	Quasi nul — c'est un traducteur, pas un modèle de QA, et le prompt n'a aucune instruction
NLLB fine-tuné	0,3176	0,2484	Bond considérable après fine-tuning, mais reste sous le TF-IDF

→ Le notebook de démarrage recommandait mT5 pour aller plus loin — c'est ce qui a motivé notre premier modèle.

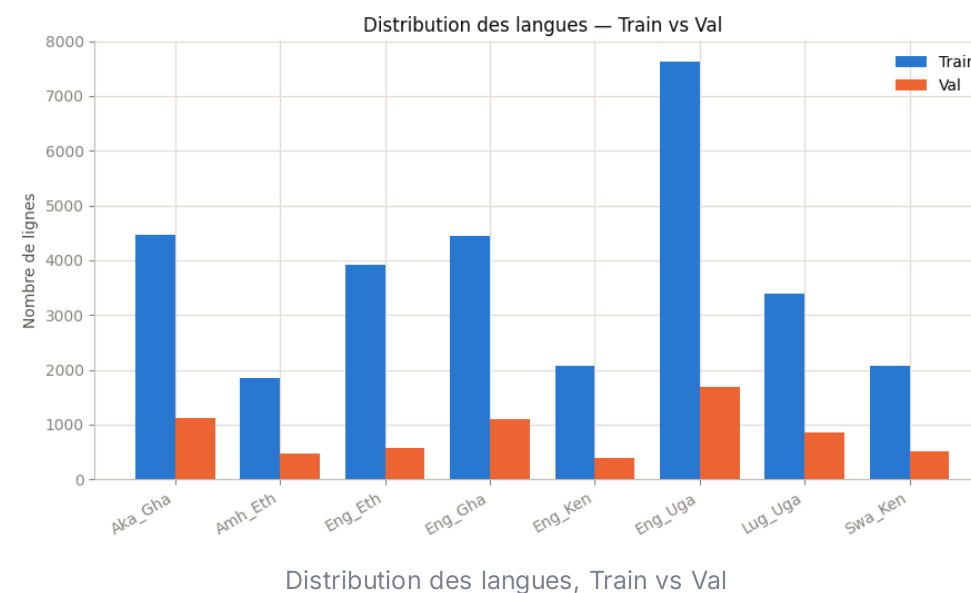
EXPLORATION DES DONNÉES

Un jeu de données propre, mais déséquilibré

29 815

questions d'entraînement

- Aucune valeur manquante, aucun doublon d'ID — pas de nettoyage structurel nécessaire.
- **Eng_Uga** pèse plus de **4×** la langue la moins représentée (Amh_Eth).
- Tokeniseurs testés : jusqu'à **3 tokens/mot** pour le luganda et l'amharique, contre 1,2 pour l'anglais.



MODÈLE 1

mT5-small — comment on l'a entraîné

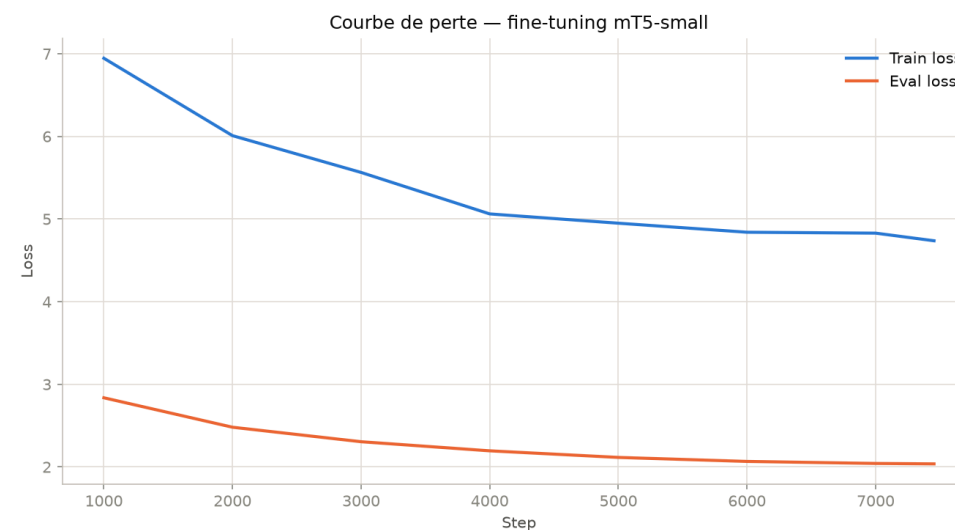
300_M

paramètres, **tous** mis à jour (full fine-tuning)

2h33

d'entraînement — 2 epochs, 7 454 steps, GPU T4

- Optimiseur **Adafactor** — cohérent avec le pré-entraînement de toute la famille T5.
- Modèle compact : abordable sur un seul GPU Colab, contrairement à Gemma E2B.



Aucune remontée de la loss d'éval sur les 7 454 steps

MODÈLE 1 — RÉSULTATS

mT5 : des résultats solides, sans surprise sur l'amharique

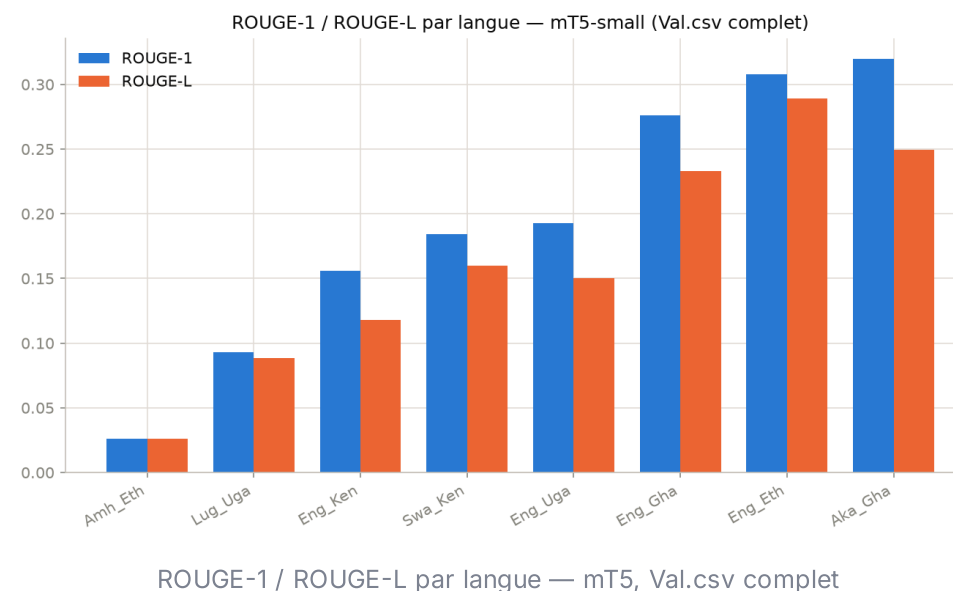
0,21 ROUGE-1

sur le jeu de validation complet (6 686 ex.)

0,17 ROUGE-L

amharique = langue la plus faible (0,026)

Écart d'un facteur **~12** entre la meilleure langue (Aka_Gha) et la pire (Amh_Eth) — le script Ge'ez, très éloigné des scripts latins, est probablement sous-représenté dans le pré-entraînement.



MODÈLE 2

Et si on passait à un modèle beaucoup plus grand ?

5,13_{Md}

paramètres Gemma E2B — 17× mT5

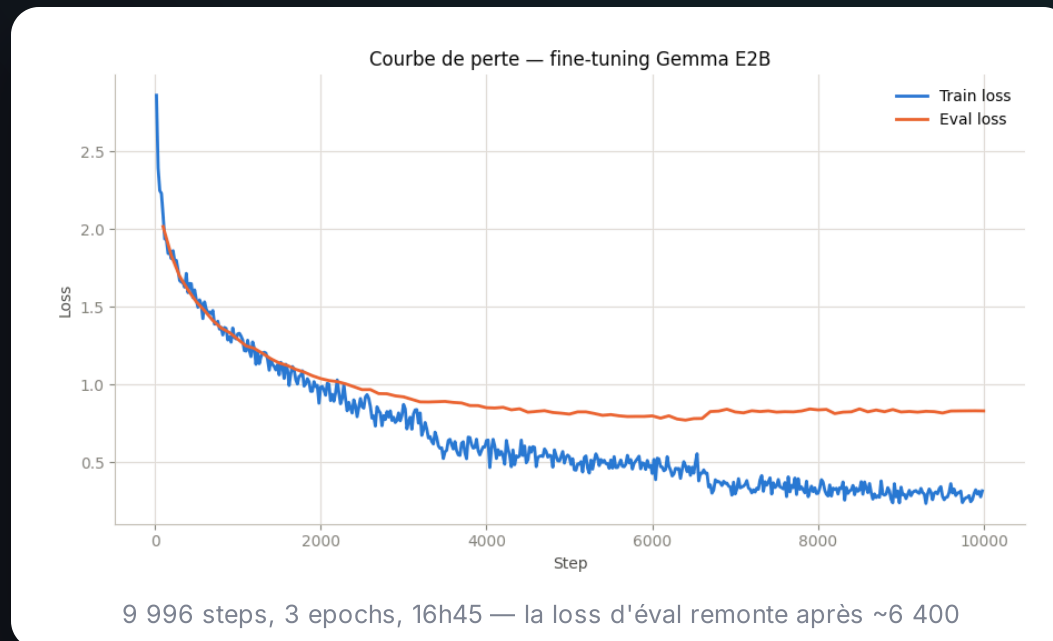
0,58%

paramètres réellement entraînés (29,9M, via LoRA)

16h45

d'entraînement — 6,5× plus long que mT5

- Fine-tuning complet hors de portée des GPU disponibles — **quantification 4-bit + LoRA** (r=16) à la place.



MODÈLE 2 — RÉSULTATS

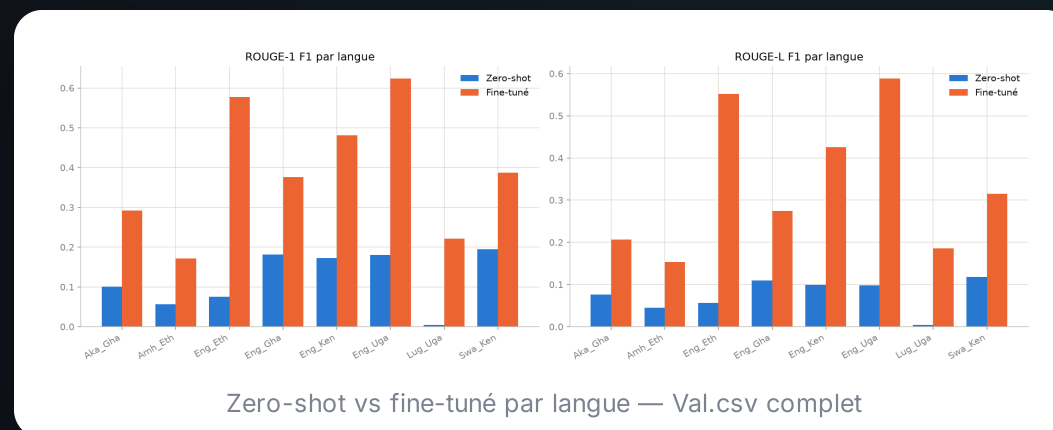
Un gain massif après fine-tuning

0,13
ROUGE-1
zero-shot → **0,41**
fine-tuné
(×3,2)

0,08
ROUGE-L
zero-shot → **0,36**
fine-tuné
(×4,5)

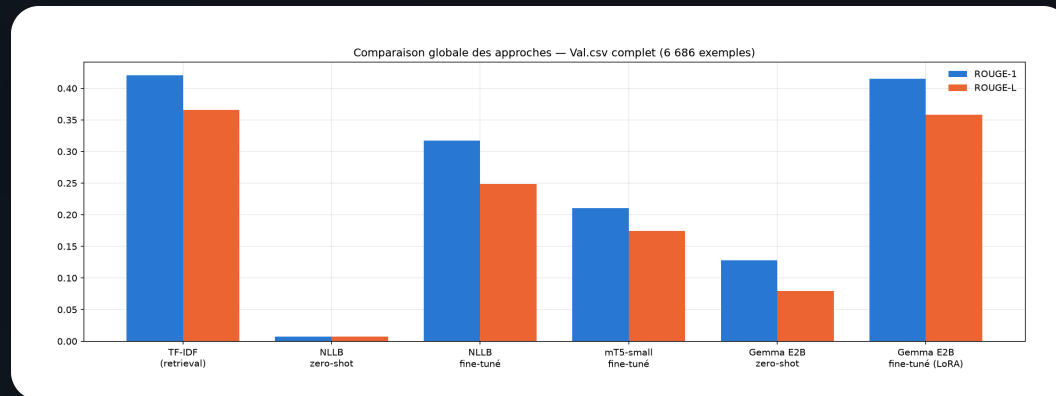
DÉCOUVERTE MARQUANTE

En zero-shot, le luganda **dégénère en boucle** (« n'okwela n'okwela... », ROUGE-1 = 0,005) — corrigé par le fine-tuning (→ 0,22).



COMPARAISON FINALE

Gemma fine-tuné : le meilleur modèle génératif



VERDICT

Le TF-IDF affiche un score plus haut, mais il **recopie** — il ne génère rien. Entre les approches génératives, **Gemma devance NLLB (+0,10) et mT5 (+0,20) sur ROUGE-1.**

- Coût : **6,5×** plus de temps d'entraînement pour **17×** plus de paramètres
- mT5 reste le choix le plus léger à déployer/réentraîner

DIFFICULTÉS TECHNIQUES

Ce qu'on a dû résoudre en cours de route

- **PEFT ne reconnaissait pas** la classe `Gemma4ClippableLinear` de Gemma — déballage vers `Linear4bit` avant d'attacher LoRA.
- **4 000+ générations perdues** après une déconnexion Colab (rien n'était sauvegardé) — checkpoint JSONL résumable, plus jamais qu'un batch perdu.
- **3 GPU différents** selon les besoins mémoire : T4 → L4 (bf16, QLoRA) → A100 80 Go (fusion de l'adaptateur en pleine précision).

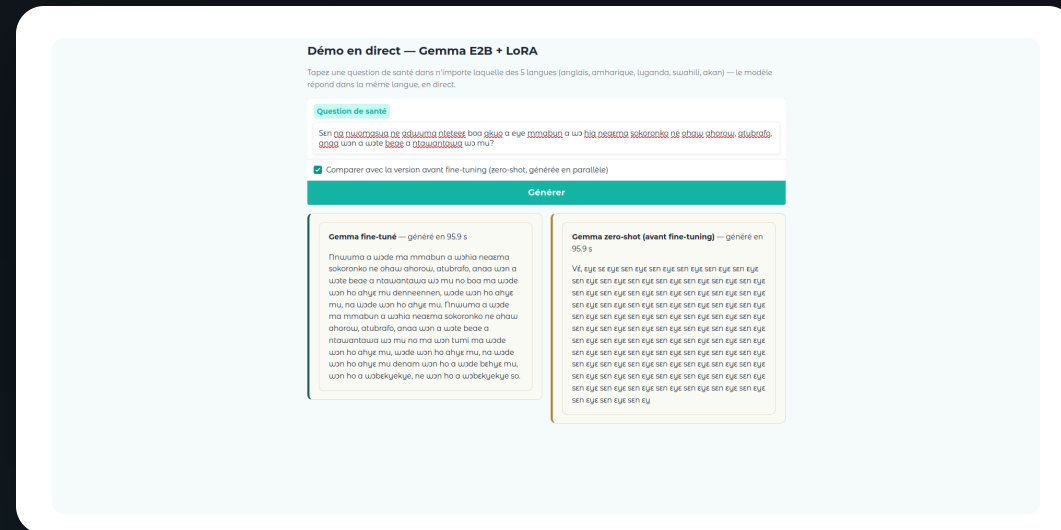
CONCLUSION

Ce que nous retenons

- Deux pipelines complets développés, partant d'une baseline Zindi plafonnant à 0,32/0,25 — **largement dépassée par Gemma** (0,41/0,36).
- **Amharique et luganda** restent structurellement les langues les plus difficiles, sur les 4 approches testées.
- Recommandations : **early stopping** pour Gemma, cibler les langues faibles en priorité, compléter ROUGE par une évaluation humaine.

NOTRE INTERFACE DE TEST

Manipuler directement Gemma E2B fine-tuné



- Interface Gradio construite pour tester le modèle : réponse générée en **streaming** (mot par mot)
- Fine-tuné et zero-shot générés **en parallèle**, sur deux modèles séparés en mémoire
- Testée en conditions réelles tout au long du développement

CE QUE CE TEST A RÉVÉLÉ

Le bug de répétition, encore une fois

EXEMPLE OBSERVÉ PENDANT LE TEST

En akan, le modèle **zero-shot** boucle sur un mot répété (« sen eye sen eye... ») au lieu de répondre — la version **fine-tunée**, elle, répond correctement à la même question.

- Même mode de dégénérescence déjà observé sur le **luganda** en section 5.4 du rapport (ROUGE-1 = 0,005 en zero-shot)
- Confirmation sur une **deuxième langue**, en conditions réelles, que le fine-tuning corrige ce problème de façon fiable



Merci — Questions ?

Andil Mchangama · Nasma Youssouf M'madi · Abdoulaye Diouf · Harouna Mbaye · Hamidou Mbodji

